

Banka Java Mülakatı

Eksik Konular Çalışma Notu

Batur için kişiselleştirilmiş çalışma dokümanı

Gerçek hayat senaryoları ile anlatım

Bu doküman, mülakat simülasyonunda eksik kaldığın konuları banka uygulamalarından örneklerle anlatmak için hazırlandı.

Her bölümde önce gerçek bir senaryo, sonra teknik açıklama, ardından kod örneği ve son olarak akılda kalması gereken nokta var.

Süre: 2-3 saatte rahatlıkla bitirebilirsin.

Sıralama: İstediğin bölümden başlayabilirsin, hepsi bağımsız.

İçindekiler

1	Eşzamanlılık (Concurrency)	Para çekme örneğiyle synchronized, volatile, Lock
2	@Transactional Gerçekte Nasıl Çalışır?	Proxy, Propagation, çalışmadığı durumlar
3	Dağıtık Transaction'lar	Para gitti karşıya ulaşmadı — Saga, Outbox, Idempotency
4	Stream API'yi Pratikte Kullanmak	groupingBy ve collect'leri ezberle
5	ArrayList vs LinkedList Doğru Hangisi	Hangisi ne zaman, doğru Big-O'lar
6	JVM Belleği ve Garbage Collection	Heap, Stack, Generations sade dille
7	Builder Pattern	15 parametrelili constructor yerine
8	Spring Derinlemesine	@Repository sihri, AOP proxy, Bean lifecycle
9	REST API ve HTTP Status Code'ları	200 ne, 201 ne, 409 ne zaman
10	Production Incident Yönetimi	Banka kültürü: tek başına SQL çalıştırma!
11	Mülakatta Anlatabileceğin Hikayeler	ARCUS deneyimini banka diline çevir

1. Eşzamanlılık (Concurrency)

Mülakatta synchronized'i "lokal değişkenler için" demen ve volatile'ı deadlock'la bağdaştırman temel bir kavram karışıklığıydı. Bu bölüm kafanda netleştiresin.

1.1. Önce Problemi Anlayalım: Race Condition Nedir?

□ *Gerçek Hayat Senaryosu:*

Bir müşterinin hesabında 1000 TL var. Eşi ATM'den 800 TL çekiyor, aynı saniyede müşteri de internet bankacılığından 600 TL para transfer ediyor. Sistemde şu kod var:

```
public void paraCek(int miktar) {
    int mevcut = hesap.getBakiye(); // Adım 1: Bakiyeyi oku
    if (mevcut >= miktar) { // Adım 2: Yeterli mi kontrol et
        hesap.setBakiye(mevcut - miktar); // Adım 3: Yeni bakiyeyi yaz
    }
}
```

Şimdi şunu hayal et: ATM thread'i ile internet bankacılığı thread'i tam aynı anda Adım 1'i çalıştırıyor. İkisi de bakiyeyi 1000 TL olarak okuyor. İkisi de Adım 2'de "evet yeterli" diyor. ATM 200 TL'ye düşürüyor, internet bankacılığı 400 TL'ye düşürüyor. Banka 1400 TL kaybetti, üstelik hesabın son bakiyesi sıra meselesine bağlı. Bu race condition'dır.

□ **Akılda Kalması Gereken: Birden fazla thread paylaşılan bir veriyi (instance/static field, koleksiyon, DB satırı) aynı anda değiştiriyorsa SORUN VAR. Lokal değişkenlerde sorun YOK çünkü her thread'in kendi stack'inde, paylaşılmıyor.**

1.2. synchronized — "Bu Kapıdan Aynı Anda Sadece Bir Kişi Geçer"

synchronized aslında çok basit bir şey yapar: işaretlediğin bloğa veya metoda aynı anda sadece bir thread girebilir. Diğerleri kapıda bekler.

Banka şubesindeki kasayı düşün. Kasanın içinde para sayan tek bir görevli var. Müşteriler tek tek giriyor, işini bitiren çıkıyor, sıradaki giriyor. İkisi aynı anda giremiyor. synchronized bu kapıyı koyar.

```
public synchronized void paraCek(int miktar) {
    int mevcut = hesap.getBakiye();
    if (mevcut >= miktar) {
        hesap.setBakiye(mevcut - miktar);
    }
}
// Artık iki thread aynı anda BU metoda giremez.
```

Üç önemli detay:

1) Hangi nesne üzerinde kilit? synchronized bir instance method'a konulursa this üzerinde, static method'a konulursa Class nesnesi üzerinde kilitlenir. Yani aynı nesnenin iki farklı synchronized metoduna iki thread aynı anda giremez (aynı kilit).

2) Synchronized block daha iyidir: Tüm metodu kilitlemek yerine sadece kritik bölümü kilitle:

```
public void paraCek(int miktar) {  
    // Bu kısım paralel çalışabilir, kilitlemeye gerek yok  
    log.info("Para çekme isteği geldi: {}", miktar);  
  
    synchronized(hesap) { // Sadece hesap güncellemesi kilitli  
        if (hesap.getBakiye() >= miktar) {  
            hesap.setBakiye(hesap.getBakiye() - miktar);  
        }  
    }  
  
    bildirimGonder(); // Bu da kilit dışında olabilir  
}
```

3) synchronized YAVAŞTIR. Banka gibi yüksek trafikli sistemlerde her şeyi synchronized'le çözmek sistemi öldürür. Saniyede 5000 istek geliyorsa ve hepsi aynı metoda kilitleniyorsa kuyruk oluşur, response time 30 saniyeye çıkar.

⚠ Dikkat: Mülakatta "lokal değişkene synchronized" deme. Lokal değişken zaten thread'e özeldir. synchronized paylaşılan state için.

1.3. volatile — "Bu Değişkeni Cache'leme, Hep Ana Bellekten Oku"

□ Gerçek Hayat Senaryosu:

Bir batch job thread'i çalışıyor, 1 milyon kayıt işliyor. Yöneticisi gelip "Bunu durdur" demek istiyor. Stop butonuna basıyor. Stop butonu şu kodu çalıştırıyor:

```
class BatchJob extends Thread {  
    private boolean durdur = false;  
  
    public void run() {  
        while (!durdur) {  
            kayıtIsle();  
        }  
    }  
  
    public void durdurBatch() {  
        durdur = true; // Stop butonu bunu çağırıyor  
    }  
}
```

Sorun: Batch thread'i "durdur" değişkenini kendi CPU cache'inde tutar. Stop butonuna bastığında durdur = true ana belleğe yazılır ama batch thread'i bunu görmez, kendi cache'indeki "false" değeriyle sonsuza kadar çalışmaya devam eder. Batch hiç durmaz!

Çözüm:

```
private volatile boolean durdur = false;
```

volatile JVM'e şunu der: "Bu değişkeni cache'leme. Her okuma ve yazma DOĞRUDAN ana bellekten/ana belleğe olsun." Böylece bir thread değiştirir değiştirmez diğer thread'ler görür. Buna visibility (görünürlük) denir.

⚠ Dikkat: volatile DEADLOCK ile ilgili değildir. volatile sadece görünürlük sağlar, atomicity sağlamaz. counter++ volatile olsa bile race condition'a açıktır çünkü o aslında 3 işlem: oku, artır, yaz. Bunlar arasında başka thread girebilir.

□ **Akılda Kalması Gereken: synchronized = mutual exclusion (aynı anda tek thread) + visibility**

volatile = sadece visibility (görünürlük), atomicity YOK

volatile ne zaman? Flag değişkenleri (boolean dur/devam), config bayrakları gibi tek yazıcı + birden fazla okuyucu senaryolarında.

1.4. ReentrantLock — Akıllı synchronized

ReentrantLock synchronized'in yapabildiği her şeyi yapar, ama daha fazlasını da yapar. Banka senaryolarında çok işine yarar.

□ *Gerçek Hayat Senaryosu:*

Bir müşteri internet bankacılığında havale yapıyor. Karşı tarafın hesabı kilitli (başka bir transaction güncelliyor). Müşteriyi 30 saniye bekletmek istemiyorsun. "En fazla 2 saniye bekle, alamazsan vazgeç ve müşteriye 'tekrar deneyin' de" demek istiyorsun. synchronized bunu yapamaz — bekleme süresi koyamazsın.

```
private final ReentrantLock lock = new ReentrantLock();

public boolean paraTransferi(Hesap karsi, int miktar) {
    try {
        // En fazla 2 saniye bekle, kilit alamazsan false dön
        if (lock.tryLock(2, TimeUnit.SECONDS)) {
            try {
                karsi.bakiyeArttir(miktar);
                return true;
            } finally {
                lock.unlock(); // ÇOK ÖNEMLİ: finally'de unlock yap
            }
        } else {
            // 2 saniye bekledik, kilit alamadık
            return false; // Müşteriye "yoğunluk var, tekrar deneyin"
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        return false;
    }
}
```

ReentrantLock'un synchronized'a göre üstünlükleri:

- tryLock(timeout): Süre koyabilirsin (yukarıdaki örnek)
- lockInterruptibly(): Bekleyen thread'i interrupt edebilirsin
- Fair lock: new ReentrantLock(true) ile FIFO sırası garanti. synchronized'da hangi thread'in kilit alacağı belirsiz, ReentrantLock'ta isteğe bağlı olarak sırayla.
- Condition: Belirli koşullarda thread'leri bekletip uyandırabilirsin

⚠ Dikkat: ReentrantLock'ta unlock() çağırmaı UNUTMA. synchronized'da bunu JVM otomatik yapar (block bitince), ReentrantLock'ta sen yapmazsan sonsuza kadar kilitli kalır. Her zaman try-finally içinde unlock yap.

1.5. AtomicInteger ve Arkadaşları — Lock'suz Sayma

Sayaç tutuyorsun: "Bugün kaç para çekme işlemi oldu?" gibi. Her işlemde sayac++ yapıyorsun. Bu race condition'a açık (yukarıda anlattık). synchronized koyabilirsin ama sadece bir sayı için synchronized koymak israf.

```
// KÖTÜ - Race condition var
private int paraCekmeSayisi = 0;
public void islemTamamlandi() {
    paraCekmeSayisi++; // Atomik değil!
}

// İYİ - AtomicInteger kullan
private AtomicInteger paraCekmeSayisi = new AtomicInteger(0);
public void islemTamamlandi() {
    paraCekmeSayisi.incrementAndGet(); // Atomik
}
```

AtomicInteger, AtomicLong, AtomicReference gibi sınıflar CAS (Compare-And-Swap) denen donanım seviyesinde bir mekanizmayla çalışır. Lock kullanmaz, çok hızlıdır. Sayaç, flag, referans güncelleme gibi tek değişkenli işlemlerde ideal.

❏ **Akılda Kalması Gereken: Tek değişken için lock alma, AtomicXxx kullan. Lock paralelliği öldürür, AtomicXxx donanım desteğiyle çalışır, çok daha hızlı.**

1.6. ConcurrentHashMap — Çok Thread'li HashMap

HashMap thread-safe değildir. İki thread aynı anda put yaparsa map bozulabilir, hatta sonsuz döngüye girebilir (Java 7 öncesi meşhur bug). Banka uygulamalarında in-memory cache çok kullanılır, bunlar genelde paylaşılır.

```
// KÖTÜ - thread-safe değil
Map<String, Hesap> cache = new HashMap<>();

// ESKİ ÇÖZÜM - Tüm map'i kilitler, yavaş
Map<String, Hesap> cache = Collections.synchronizedMap(new HashMap<>());
```

```
// İYİ - Bucket-level lock, çok daha hızlı  
Map<String, Hesap> cache = new ConcurrentHashMap<>();
```

ConcurrentHashMap tüm map'i kilitlemez, sadece üzerinde çalıştığı bucket'ı kilitler. Aynı anda farklı bucket'lara yazan thread'ler birbirini beklemez. Banka uygulamalarında session cache, parametre cache, exchange rate cache gibi yerlerde sürekli karşına çıkar.

2. @Transactional Gerçekte Nasıl Çalışır?

Mülakatta "ya hepsi ya hiç" demen doğru ama yetersizdi. Bu konu bankada her gün karşına çıkacak — bu yüzden derinleştelim.

2.1. Sahne Arkasında Ne Oluyor? Proxy Sihri

Sen şu kodu yazıyorsun:

```
@Service
public class HavaleServisi {

    @Transactional
    public void havaleYap(String gonderen, String alici, int miktar) {
        hesapRepo.paraCek(gonderen, miktar);
        hesapRepo.paraYatir(alici, miktar);
    }
}
```

Sonra başka bir yerden çağırıyorsun:

```
@Autowired
private HavaleServisi havaleServisi;

havaleServisi.havaleYap("A123", "B456", 1000);
```

Burada Spring'in büyük sihri devreye giriyor: Sen "havaleServisi" dediğinde Spring sana gerçek HavaleServisi nesnesini değil, onun bir proxy'sini veriyor. Proxy senin sınıfından türetilmiş bir alt sınıf gibi. Her metot çağrısını yakalıyor. Şöyle düşün:

```
// Spring'in arka planda yaptığı şey (basitleştirilmiş)
public class HavaleServisi$Proxy extends HavaleServisi {

    @Override
    public void havaleYap(String gonderen, String alici, int miktar) {
        try {
            transactionManager.begin(); // 1. Transaction başlat
            super.havaleYap(gonderen, alici, miktar); // 2. Gerçek metodu çağır
            transactionManager.commit(); // 3. Başarılıysa commit
        } catch (RuntimeException e) {
            transactionManager.rollback(); // 4. Hata varsa rollback
            throw e;
        }
    }
}
```

❏ **Akılda Kalması Gereken: @Transactional işlemini Spring yapmaz, Spring'in oluşturduğu PROXY yapar. Bu, sonraki problemleri anlamanın anahtarı.**

2.2. "@Transactional Koymuşum Ama Çalışmıyor!" — Klasik Tuzaklar

Tuzak 1: Self-Invocation (Aynı Sınıftan Kendine Çağrı)

```
@Service
public class HavaleServisi {

    public void topluHavale(List<HavaleIstegi> istekler) {
        for (HavaleIstegi istek : istekler) {
            this.havaleYap(istek); // ❌ this. ile çağırdın!
        }
    }

    @Transactional
    public void havaleYap(HavaleIstegi istek) {
        // Bu @Transactional ÇALIŞMAYACAK!
        hesapRepo.paraCek(...);
        hesapRepo.paraYatir(...);
    }
}
```

Neden çalışmaz? Yukarıda gördük: @Transactional'ı proxy yapar. Ama this.havaleYap() dediğinde proxy'i değil, kendi nesnenin gerçek metodunu çağırıyorsun. Proxy devre dışı, transaction başlamıyor.

Çözüm: Ya havaleYap'ı farklı bir bean'e koy ve oradan @Autowired ile çağır, ya da kendi proxy'ne erişip ondan çağır:

```
@Service
public class HavaleServisi {

    @Autowired
    private HavaleServisi self; // Spring kendi proxy'ini inject eder

    public void topluHavale(List<HavaleIstegi> istekler) {
        for (HavaleIstegi istek : istekler) {
            self.havaleYap(istek); // ❌ proxy üzerinden çağrı
        }
    }

    @Transactional
    public void havaleYap(HavaleIstegi istek) { ... }
}
```

Tuzak 2: private veya final Metot

Proxy senin sınıfının alt sınıfı gibi davranıyor. Alt sınıf private ve final metotları override edemez. Bu yüzden:

```
@Transactional
private void havaleYap(...) { ... } // ❌ ÇALIŞMAZ - private

@Transactional
```

```
public final void havaleYap(...) { ... } // ❌ ÇALIŞMAZ - final

@Transactional
public void havaleYap(...) { ... } // ❌ ÇALIŞIR
```

Tuzak 3: Checked Exception'ı Rollback Sanmak

Default davranış: Spring sadece RuntimeException (unchecked) fırlatıldığında rollback yapar. Checked exception'da otomatik rollback YOK.

```
@Transactional
public void havaleYap(...) throws SQLException {
    hesapRepo.paraCek(...);
    if (yetersizBakiye) {
        throw new SQLException("Bakiye yetersiz"); // ❌ ROLLBACK YAPMAZ!
    }
    hesapRepo.paraYatir(...);
}

// Çözüm 1: rollbackFor belirt
@Transactional(rollbackFor = Exception.class)
public void havaleYap(...) throws SQLException { ... }

// Çözüm 2: Daha iyi, RuntimeException fırlat
@Transactional
public void havaleYap(...) {
    if (yetersizBakiye) {
        throw new YetersizBakiyeException("Bakiye yetersiz");
    }
}

}
```

Tuzak 4: try-catch ile Exception'ı Yutmak

```
@Transactional
public void havaleYap(...) {
    try {
        hesapRepo.paraCek(...);
        hesapRepo.paraYatir(...);
    } catch (Exception e) {
        log.error("Hata oldu", e);
        // ❌ Exception fırlatmadın, Spring "her şey yolunda" sanıyor, COMMIT yapıyor
    }
}
```

Eğer exception'ı yakalayıp yutarsan Spring rollback yapmaz çünkü exception'ı görmez. Ya tekrar fırlat (rethrow), ya da manuel rollback yap.

2.3. Propagation — "Transaction İçinde Transaction Olunca Ne Olur?"

❏ *Gerçek Hayat Senaryosu:*

Havale yapıyorsun. Havale tamamlandığında audit log tablosuna kayıt atman lazım — "X kullanıcısı şu havaleyi denedi". Ama havale fail olsa bile audit log kalmalı (güvenlik için). Yani havale rollback olsa bile audit log COMMIT olmalı.

```
@Service
public class HavaleServisi {

    @Transactional // Default: REQUIRED
    public void havaleYap(HavaleIstegi istek) {
        auditService.logla(istek); // ← Bu transaction nasıl olmalı?
        hesapRepo.paraCek(...);
        hesapRepo.paraYatir(...); // Burada fail olursa
        // havaleYap rollback olur, ama AUDIT LOG'UN KALMASI LAZIM
    }
}
```

İşte propagation burada devreye giriyor. auditService.logla'yı nasıl tanımlarsın:

```
@Service
public class AuditService {

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void logla(HavaleIstegi istek) {
        // Bu metoda girildiğinde:
        // 1. Havale transaction'ı ASKIYA ALINIR
        // 2. YENİ bir transaction başlar
        // 3. Bu commit edilir (havale fail olsa bile)
        // 4. Sonra havale transaction'ı devam eder
        auditRepo.kaydet(istek);
    }
}
```

En sık karşılaşacağın iki propagation:

Propagation	Davranış	Banka Senaryosu
REQUIRED (default)	Mevcut transaction varsa ona katılır, yoksa yeni başlatır. Her şey ya commit olur ya rollback olur.	Hesap ile hesap arasında. Her şey ya commit olur ya rollback olur.
REQUIRES_NEW	Mevcut transaction'ı askıya al, yepyeni transaction başlat. Başka yazılamaz, işlemler fail olsa bile commit olur.	Audit log işlemleri gibi bağımsız işlemler için.

Diğerleri (SUPPORTS, MANDATORY, NEVER, NESTED, NOT_SUPPORTED) daha nadir, isimlerini bilmen yeter. Mülakatta REQUIRED ve REQUIRES_NEW'i sağlam anlat.

📌 Akılda Kalması Gereken: @Transactional anahtar bilgileri:

- 1) Proxy ile çalışır — bu yüzden self-invocation çalışmaz
- 2) public metod olmalı — private, final çalışmaz
- 3) Sadece RuntimeException rollback yapar — checked için rollbackFor kullan
- 4) Exception yutma — yutarsan Spring rollback yapmaz
- 5) REQUIRES_NEW — audit log gibi bağımsız işlemler için

3. Dağıtık Transaction'lar — Bankanın En Zor Problemi

Mülakatın en kritik senaryosu buydu: "Para gitti karşıya ulaşmadı." Sen "manuel SQL ile düzeltirim" dedin — bu bankada FELAKET cevap. Burada hem konsepti, hem doğru aksiyonu öğreneceksin.

3.1. Tek DB'de ACID Var, Sistemler Arası YOK

Senin DB'nde havale yaptığında ACID garanti: para hesabından düşer, muhasebe tablosuna yazar, hepsi tek transaction'da. Ama karşı banka ayrı bir sistem. Onun DB'sine senin transaction'ın uzanmaz. Aralarında network var, network kopabilir.

□ Gerçek Hayat Senaryosu:

Havale akışı şöyle ilerliyor:

1. Senin DB: müşterinin hesabından 1000 TL düş → COMMIT
2. Karşı bankaya REST çağrısı: "Hesap B456'ya 1000 TL yatır"
3. Karşı banka: 1000 TL'yi yatırır, sana 200 OK döner

Sorun: Adım 2 ile 3 arasında network kopuyor. Karşı banka 1000 TL'yi yatırmış mı, yoksa istek hiç ulaşmamış mı? Sen bilmiyorsun. Senin hesabından para gitti, karşı tarafta var mı yok mu belirsiz.

3.2. Idempotency — "Aynı İsteği İki Kez Yollarsan İki Kez İşlem Yapma"

Idempotent demek, aynı operasyonu bir veya bin defa çağırırsan sonuç aynı demek. Banka API'lerinde olmazsa olmaz.

Yukarıdaki senaryoda sen ne yaparsın? Retry edersin:

```
// Network koptu, karşı taraftan cevap gelmedi
try {
    karsiBankaApi.paraYatir("B456", 1000);
} catch (TimeoutException e) {
    // Bilmiyoruz, ulaştı mı ulaşmadı mı?
    // Tekrar deneyelim
    karsiBankaApi.paraYatir("B456", 1000);
}
```

Problem: Ya ilk istek ulaştıysa? O zaman karşı tarafa 2000 TL yatmış olur. Çözüm: Her isteğe idempotency key (benzersiz anahtar) ekle:

```
// Her havaleye benzersiz UUID ata
String idempotencyKey = UUID.randomUUID().toString();
// Örnek: "havale-2024-03-15-a4f5b8c2"
```

```
ParaYatirIstegi istek = new ParaYatirIstegi();
istik.setIdempotencyKey(idempotencyKey);
istik.setAlici("B456");
istik.setMiktar(1000);

karsiBankaApi.paraYatir(istik);
// Timeout olursa aynı key ile tekrar dene
// Karşı banka aynı key'i gördüğünde "bu işlemi zaten yaptım" der, tekrar yapmaz
```

Karşı bankanın tarafında:

```
@PostMapping("/para-yatir")
public Response paraYatir(@RequestBody ParaYatirIstegi istek) {
    String key = istek.getIdempotencyKey();

    // Bu key'i daha önce işledim mi?
    Optional<IslemKaydi> mevcut = islemRepo.findByIdempotencyKey(key);
    if (mevcut.isPresent()) {
        // Aynı isteği geri dön, tekrar para yatırma
        return mevcut.get().toResponse();
    }

    // İlk kez geliyor, normal işlem yap
    hesapRepo.paraEkle(istik.getAlici(), istek.getMiktar());
    islemRepo.kaydet(key, istek);
    return Response.ok();
}
```

❑ **Akılda Kalması Gereken: Banka API'sinde her finansal işlem idempotent OLMALI. Idempotency key olmadan retry yapamazsın, retry yapamazsan network sorunlarında veri tutarsızlığı yaşarsın.**

3.3. Outbox Pattern — "DB Commit + Mesaj Gönderme Birlikte"

❑ *Gerçek Hayat Senaryosu:*

Sen havale yaptıktan sonra müşteriye SMS atman, mobil bildirim göndermen, muhasebeye event yollaman lazım. Şöyle yazıyorsun:

```
@Transactional
public void havaleYap(...) {
    hesapRepo.paraCek(...);
    hesapRepo.paraYatir(...);
    // DB commit edildi

    smsServisi.gonder(...); // Kafka'ya event yolla
    bildirimServisi.gonder(...);
}
```

Sorun: DB commit oldu ama Kafka çağrısı sırasında uygulama crash etti. Para gitti ama SMS gitmedi. Müşteri SMS bekliyor, gelmiyor. Ya da tam tersi: Kafka'ya yazdın, sonra DB commit fail oldu, SMS attın ama para hareketi gerçekleşmedi.

Outbox Pattern çözümü: Mesajları doğrudan Kafka'ya gönderme. Aynı DB transaction'ı içinde özel bir outbox tablosuna kaydet. Sonra ayrı bir job bu tabloyu okuyup mesajları gönderir.

```
@Transactional
public void havaleYap(...) {
    hesapRepo.paraCek(...);
    hesapRepo.paraYatir(...);

    // Mesajı doğrudan Kafka'ya YOLLAMA, outbox tablosuna YAZ
    outboxRepo.kaydet(new OutboxKaydi(
        "SMS_GONDER",
        smsPayload,
        Status.YENI
    ));

    // Hepsi tek transaction. DB commit olursa outbox kaydı da kalır.
}

// Ayrı bir scheduled job:
@Scheduled(fixedDelay = 5000)
public void outboxIsle() {
    List<OutboxKaydi> yeniler = outboxRepo.findByStatus(Status.YENI);
    for (OutboxKaydi kayit : yeniler) {
        try {
            kafkaProducer.gonder(kayit);
            outboxRepo.statusGuncelle(kayit.getId(), Status.GONDERILDI);
        } catch (Exception e) {
            // Yarın tekrar denenecek, outbox'ta kalır
        }
    }
}
```

Sonuç: DB commit olduysa outbox kaydı da var. Kafka çağrısı başarısız olursa outbox'ta kalır, sonraki retry'da gönderilir. DB ile mesaj kuyruğu arasındaki tutarsızlık çözüldü.

3.4. Saga Pattern — "Geri Almak İçin Plan Yap"

Çok adımlı, çok sistemli bir iş akışında bir adım fail olursa nasıl geri alırsın? Tek DB'de "rollback" yapardın. Ama 5 farklı sistemde 5 farklı şey yaptıysan rollback yok. Saga der ki: her ileri adım için bir geri alma adımı (compensation) yaz.

□ *Gerçek Hayat Senaryosu:*

Müşteri otel + uçak + sigorta'yı tek paket olarak ödüyor:

- 1. Otel rezervasyonu yap → başarılı*
- 2. Uçak bileti al → başarılı*

3. Sigorta yap → BAŞARISIZ

Şimdi ne yapacaksın? Otelden ve uçaktan rezervasyon iptal istemen lazım.

```
// Saga akışı
try {
    String otelId = otelServisi.rezerveEt(...); // Adım 1

    try {
        String biletId = ucakServisi.biletAl(...); // Adım 2

        try {
            sigortaServisi.sigortaYap(...); // Adım 3
        } catch (Exception e) {
            // Adım 3 fail - 2 ve 1'i geri al
            ucakServisi.biletIptal(biletId); // Compensation 2
            otelServisi.rezervasyonIptal(otelId); // Compensation 1
            throw e;
        }
    } catch (Exception e) {
        otelServisi.rezervasyonIptal(otelId); // Compensation 1
        throw e;
    }
} catch (Exception e) {
    log.error("Paket satış başarısız", e);
}
```

Önemli: Compensation'lar da fail edebilir. Bu yüzden compensation'lar da idempotent olmalı ve retry edilebilmeli. Saga frameworkleri (Axon, Camunda) bu karmaşıklığı yönetmek için var.

3.5. Reconciliation — "Her Gece Sayım Yap"

Her şeyi mükemmel yapsan bile dağıtık sistemlerde tutarsızlık olabilir. Bunun için bankalarda gece reconciliation job çalışır: senin DB'ndeki kayıtlar ile karşı sistemlerin kayıtlarını karşılaştır, fark varsa flag'le.

```
@Scheduled(cron = "0 0 2 * * *") // Her gece 02:00
public void mutabakatYap() {
    LocalDate dun = LocalDate.now().minusDays(1);

    // Senin kayıtların
    List<Havale> senin = havaleRepo.findByTarih(dun);

    // Karşı bankadan dünün listesi
    List<Havale> karsi = karsiBankaApi.getDailyTransactions(dun);

    // İki listeyi karşılaştır
    List<Havale> uyumsuz = bul Farkliliklar(senin, karsi);

    if (!uyumsuz.isEmpty()) {
        // Operasyon ekibine alarm gönder
    }
}
```

```
        alarmServisi.bildir("Mutabakat farkı: " + uyumsuz.size() + " kayıt");  
        // İnsan müdahalesi gerekecek  
    }  
}
```

Banka sistemleri ne kadar iyi yazılırsa yazılsın mutlaka reconciliation yapar. Çünkü %0.001 tutarsızlık bile milyonluk işlem hacminde büyük rakamlara denk gelir.

□ **Akılda Kalması Gereken: Dağıtık transaction için cevap setin:**

- 1) Idempotency key — retry'a hazır ol**
- 2) Outbox pattern — DB ile mesaj kuyruğu tutarlılığı**
- 3) Saga pattern — çok adımlı iş akışlarında compensation**
- 4) Reconciliation — gece sayımı, fark varsa alarm**
- 5) Circuit breaker — sürekli fail eden servisi geçici kapat (Resilience4j)**

4. Stream API'yi Pratikte Kullanmak

Mülakatta kavramı biliyordun ama yazarken karıştı. Bu bölüm tamamen kas hafızası içindir — bu kalıpları gördüğünde yazabilmen lazım.

4.1. Üç Temel Operasyon: filter, map, collect

Senaryomuz: Elimizde çalışan listesi var.

```
class Calisan {
    String ad;
    String departman;
    int maas;
    int yas;
    // getter'lar...
}
```

```
List<Calisan> calisanlar = ...;
```

filter — kosula uyanları sec:

```
// 30 yaş üstü çalışanlar
List<Calisan> yaslilar = calisanlar.stream()
    .filter(c -> c.getYas() > 30)
    .collect(Collectors.toList());

// Daha modern Java 16+
List<Calisan> yaslilar2 = calisanlar.stream()
    .filter(c -> c.getYas() > 30)
    .toList();
```

map — her elemanı dönüştür:

```
// Sadece çalışan isimlerini al
List<String> isimler = calisanlar.stream()
    .map(c -> c.getAd())
    .collect(Collectors.toList());

// Method reference ile daha temiz
List<String> isimler2 = calisanlar.stream()
    .map(Calisan::getAd)
    .toList();
```

filter + map + collect zinciri:

```
// IT departmanındaki 30 yaş altındakilerin isimleri
List<String> sonuc = calisanlar.stream()
    .filter(c -> c.getDepartman().equals("IT"))
    .filter(c -> c.getYas() < 30)
    .map(Calisan::getAd)
    .toList();
```

4.2. Toplama: sum, count, average

```
// Toplam maaş gideri
int toplam = calisanlar.stream()
    .mapToInt(Calisan::getMaas)
    .sum();

// Çalışan sayısı
long sayi = calisanlar.stream()
    .filter(c -> c.getDepartman().equals("IT"))
    .count();

// Ortalama maaş
OptionalDouble ortalama = calisanlar.stream()
    .mapToInt(Calisan::getMaas)
    .average();

double ortalamaDeger = ortalama.orElse(0);
```

4.3. groupingBy — Mülakatta Yazabilmen Lazım

Senaryo: "Departman bazında ortalama maaş hesapla" — Sen bunu yapamadın. Doğru yazılışı:

```
// Departmana göre ortalama maaş
Map<String, Double> ortalamlar = calisanlar.stream()
    .collect(Collectors.groupingBy(
        Calisan::getDepartman, // Anahtar
        Collectors.averagingInt(Calisan::getMaas) // Değer hesabı
    ));

// Sonuç:
// { "IT" → 25000.0, "Muhasebe" → 18000.0, "Pazarlama" → 22000.0 }
```

Avnı kalıbın baska varvasvonları:

```
// 1. Departmana göre çalışan listesi
Map<String, List<Calisan>> dGore = calisanlar.stream()
    .collect(Collectors.groupingBy(Calisan::getDepartman));

// 2. Departmana göre çalışan sayısı
Map<String, Long> sayilar = calisanlar.stream()
    .collect(Collectors.groupingBy(
        Calisan::getDepartman,
        Collectors.counting()
    ));

// 3. Departmana göre toplam maaş
Map<String, Integer> toplamlar = calisanlar.stream()
    .collect(Collectors.groupingBy(
        Calisan::getDepartman,
        Collectors.summingInt(Calisan::getMaas)
    ));

// 4. Departmana göre en yüksek maaşlı çalışan
Map<String, Optional<Calisan>> enIyiler = calisanlar.stream()
    .collect(Collectors.groupingBy(
```

```
        Calisan::getDepartman,  
        Collectors.maxBy(Comparator.comparingInt(Calisan::getMaas))  
    ));
```

4.4. Banka Senaryosu: Müşteri İşlem Analizi

□ Gerçek Hayat Senaryosu:

Bir günün işlem listesini analiz edeceksin. Hangi şubeden ne kadar para hareketi olmuş, en çok hangi şube işlem yapmış, ortalama işlem tutarı ne?

```
class Islem {  
    String subeKodu;  
    String tip; // "HAVALE", "EFT", "ATM"  
    BigDecimal tutar;  
    LocalDateTime tarih;  
}  
  
List<Islem> bugun = islemRepo.bugunkuIslemler();  
  
// 1. Şube bazında toplam tutar  
Map<String, BigDecimal> subeTutarlari = bugun.stream()  
    .collect(Collectors.groupingBy(  
        Islem::getSubeKodu,  
        Collectors.reducing(BigDecimal.ZERO, Islem::getTutar, BigDecimal::add)  
    ));  
  
// 2. Şube + İşlem tipi bazında sayı (iç içe gruplama)  
Map<String, Map<String, Long>> detayli = bugun.stream()  
    .collect(Collectors.groupingBy(  
        Islem::getSubeKodu,  
        Collectors.groupingBy(  
            Islem::getTip,  
            Collectors.counting()  
        )  
    ));  
// { "S001" → { "HAVALE" → 45, "EFT" → 23 }, "S002" → ... }  
  
// 3. En çok işlem yapan ilk 5 şube  
List<Map.Entry<String, Long>> top5 = bugun.stream()  
    .collect(Collectors.groupingBy(Islem::getSubeKodu, Collectors.counting()))  
    .entrySet().stream()  
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed())  
    .limit(5)  
    .toList();
```

□ **Akılda Kalması Gereken: Bu kalıpları aklında tut, mülakatta her zaman çıkar:**

- **filter → map → toList():** liste filtrele ve dönüştür
- **mapToInt → sum/average:** sayısal toplam/ortalama
- **Collectors.groupingBy(anahtar, downstream):** gruplama + agregasyon

- **Downstream'ler:** `counting()`, `summingInt()`, `averagingInt()`, `maxBy()`, `reducing()`

5. ArrayList vs LinkedList — Doğru Karne

Mülakatta complexity'leri karıştırdın. Bu basit ama önemli — doğru bilmek lazım.

5.1. Doğru Time Complexity Tablosu

Operasyon	ArrayList	LinkedList	Açıklama
get(index)	$O(1)$	$O(n)$	Array'de direkt indeks, list'te gezme
add(eleman) sona ekle	$O(1)^*$ amortized	$O(1)$	* Resize gerekirse $O(n)$
add(0, eleman) başa ekle	$O(n)$	$O(1)$	Array'de kaydırma, list'te head güncelle
add(index, eleman) ortaya ekle	$O(n)$	$O(n)$	Her ikisi de gezme/kaydırma
remove(index)	$O(n)$	$O(n)$	Array'de kaydırma, list'te gezme
remove(eleman) ilk eşleşeni	$O(n)$	$O(n)$	Önce bulmak için gezme
contains(eleman)	$O(n)$	$O(n)$	Tüm listeyi gezer
size()	$O(1)$	$O(1)$	İkisi de boyutu hazır tutar
Bellek tüketimi	Düşük	Yüksek	LinkedList'te her node prev/next için ekstra alan

5.2. Hangi Durumda Hangisini?

Cevap: %95 ihtimalle ArrayList kullan. Şaşırtıcı ama doğru. Sebepleri:

- 1) CPU cache locality: ArrayList memory'de yan yana, CPU bir blok okuduğunda sonraki elemanlar zaten cache'de. LinkedList'in node'ları memory'de dağınık, her get bir cache miss. Pratikte LinkedList neredeyse hep daha yavaş.
- 2) Bellek: LinkedList'te her eleman için 24-40 byte ekstra (prev, next, item referansları). 1 milyon elemanlık liste tutarsan 30+ MB ekstra RAM.
- 3) iterator() performansı: Pratikte zaten çoğu zaman indeksle değil iterator/foreach ile geziyorsun, ikisinin de $O(n)$.

□ İyi Pratik: Banka uygulamalarında: hesap hareketleri listesi, müşteri listesi, parametre listesi — her zaman ArrayList. LinkedList'i en son ne zaman kullandığını hatırlamayanlar var. Mülakatçıya "genelde ArrayList, çok özel durumda Deque için ArrayDeque" demen yeter.

LinkedList'in tek mantıklı kullanımı: Çok sık başa/sona ekleme ve silme yapılan Queue/Deque senaryosu. Ama onun için bile ArrayDeque daha hızlıdır.

□ **Akılda Kalması Gereken:** Mülakatta sana "LinkedList ne zaman kullanırsın?" derlerse şu cevap işini görür: "Pratikte hemen hiç kullanmıyorum. Queue/Deque ihtiyaçlarımda bile ArrayDeque daha performanslı. LinkedList'in $O(1)$ başa ekleme avantajını, ArrayList'in cache locality avantajı kapatıyor."

6. JVM Belleği ve Garbage Collection

Mülakatta stack/heap'in LIFO/FIFO mantığını karıştırdın ve generation'ları yanlış anlattın. Sade dille netleştiririm.

6.1. Stack ve Heap — Farklı Amaçlar

	Stack	Heap
Ne tutar?	Metot çağrı zinciri, lokal primitive değişkenler, nesne REFERANSLARI	Tüm nesneler (new ile yaratılan), instance field'lar, array'ler
Erişim deseni	LIFO (Last In First Out): son çağrılan metot ilk biter	Dağınık erişim, sırasız
Thread başına?	Her thread'in KENDİ stack'i var (paylaşılmaz)	TÜM thread'ler aynı heap'i paylaşır (thread-safety problemleri burada)
Yönetim	Otomatik: metot bitince temizlenir	GC tarafından temizlenir
Boyut	Küçük (genelde 256KB-1MB), StackOverflowError'ın kaynağı	Büyük (-Xmx ile ayarlanır, banka uygulamasında GB'larca)

Somut örnek:

```
public void havaleYap() { // ← Bu metot çağrılınca Stack'e frame eklenir
    int miktar = 1000; // ← STACK'te (primitive)
    Hesap hesap = new Hesap(); // ← Nesne HEAP'te, referans STACK'te
    String aciklama = "EFT"; // ← String literal pool (heap içinde)

    paraCek(miktar); // ← Yeni frame eklenir
    // paraCek bitince frame stack'ten pop edilir

    // hesap referansı stack'te ama nesnesi heap'te
    // Bu metot bitince:
    // - Stack frame pop edilir
    // - hesap referansı kaybolur
    // - Heap'teki Hesap nesnesi artık unreachable
    // - Sonraki GC turunda silinir
}
```

⚠ Dikkat: Stack LIFO'dur (son giren ilk çıkar — son çağrılan metot ilk return olur). Heap'in LIFO/FIFO ile alakası yoktur — heap dinamik bir bellek havuzudur, sırasız erişilir. Mülakatta bunu söylersen kafa karışıklığı yaratır.

6.2. Generational GC — "Genç Ölür, Yaşlı Kalır" Hipotezi

Temel gözlem: Java'da yaratılan nesnelerin çoğu çok kısa ömürlüdür. Bir HTTP isteği geldiğinde DTO, Request, Response, geçici listeler yaratılır — istek bittiğinde hepsi çöp

olur. Sadece çok azı uzun ömürlüdür (cache, uzun yaşayan singleton'lar).

JVM bu gözlemden yararlanır: yeni nesneler için ayrı bir bölge, uzun ömürlüler için ayrı bölge. İkisinde farklı GC algoritmaları kullanır.

Heap iki ana bölgeye ayrılır:

1) Young Generation (Genç Nesil)

Her yeni new ile yaratılan nesne buraya gelir. İçinde:

- Eden: İlk doğdukları yer
- Survivor 0 ve Survivor 1: İlk GC turunu atlatanların gittiği yer

Eden dolduğunda Minor GC tetiklenir. Çok hızlıdır (genelde milisaniyeler). Hayatta kalan nesneler Survivor'a kopyalanır, Eden tamamen temizlenir.

2) Old Generation (Yaşlı Nesil / Tenured)

Young'da birkaç GC turunu atlatan nesneler buraya promote edilir. Genelde uzun yaşayan nesneler: cache'ler, session bilgileri, singleton bean'ler.

Old dolduğunda Major GC (Full GC) tetiklenir. Bu çok daha pahalıdır, uygulamanın yüzlerce milisaniye, hatta saniyeler boyunca durmasına neden olabilir ("stop-the-world" pause).

⚠ Dikkat: Mülakatta "Old Generation static değişkenleri tutar" deme — YANLIŞ. Static değişkenler ve class metadata'sı Metaspace'te (Java 8 öncesi PermGen'de). Old Gen, uzun yaşayan instance nesnelerini tutar.

6.3. Banka Senaryosu: Bir Memory Leak Hikayesi

□ Gerçek Hayat Senaryosu:

Bir bankacılık servisi gece sorunsuz çalışıyor, gündüz yavaşlamaya başlıyor. Akşama doğru response time'lar 5 saniyeye çıkıyor. Restart edince düzeliyor. Ertesi gün aynı senaryo. Klasik bir memory leak.

Sorun şu kodda:

```
@Service
public class MusteriCache {

    // □ Sınırsız büyüyen cache, hiç temizlenmiyor
    private static Map<String, Musteri> cache = new HashMap<>();

    public Musteri musterigetir(String id) {
        return cache.computeIfAbsent(id, k -> dbdenCek(k));
    }
}
```

Her gün milyonlarca müşteri sorgulandıkça map sürekli büyüyor. Old Gen'e yığılıyor. Full GC her seferinde daha uzun sürüyor çünkü tarama maliyetli. Sonunda

OutOfMemoryError.

Doğru vaklasım:

```
// ☐ Caffeine cache - boyut + süre sınırı
@Service
public class MusteriCache {

    private final Cache<String, Musteri> cache = Caffeine.newBuilder()
        .maximumSize(10_000) // En fazla 10K kayıt
        .expireAfterWrite(Duration.ofMinutes(30)) // 30dk sonra silinir
        .build();

    public Musteri musterigetir(String id) {
        return cache.get(id, this::dbdenCek);
    }
}
```

6.4. GC Algoritmaları — Hangisi Ne İşe Yarar?

Algoritma	Özellik	Ne Zaman
Serial GC	Tek thread, basit. Stop-the-world uzun.	Küçük uygulamalar
Parallel GC	Birden fazla thread, throughput'u artırır.	Batch işleri, raporlama
G1 (Garbage First)	Heap'i bölgelere ayırır, predictable pause	Modern uygulamaların çoğu, banka servisleri
ZGC, Shenandoah	Çok düşük latency (<10ms pause), Java 11+	Yüksek SLA gereken sistemler, ödeme sistemleri

☐ **Akılda Kalması Gereken: Mülakatta GC için bu üç şeyi söylersen yeterli:**

- 1) Young → Old promotion, generational hipoteze dayanır
- 2) Minor GC hızlıdır (Young), Major GC pahalıdır (Old, stop-the-world)
- 3) Modern uygulamada G1 default, ultra-low-latency için ZGC tercih edilir

7. Builder Pattern — "15 Parametrelî Constructor Yerine"

Bunu hiç bilmemen büyük eksiklikti. Çok kullanılan, basit bir pattern. Banka projelerinde sürekli karşına çıkar.

7.1. Problemi Görelim

□ *Gerçek Hayat Senaryosu:*

Bir Hesap nesnesi yaratmak istiyorsun. 12 alanı var: ad, soyad, tcNo, telefon, email, hesapTipi, paraBirimi, baslangicBakıyesi, sube, vadeTipi, faizOrani, kart...

```
// Telescoping Constructor anti-pattern - KÖTÜ
public class Hesap {
    public Hesap(String ad, String soyad, String tcNo) { ... }
    public Hesap(String ad, String soyad, String tcNo, String tel) { ... }
    public Hesap(String ad, String soyad, String tcNo, String tel, String email) {
    ... }
    // ... 12 farklı constructor
    public Hesap(String ad, String soyad, String tcNo, String tel, String email,
                  String hesapTipi, String paraBirimi, int baslangic, String sube,
                  String vadeTipi, double faiz, String kartTipi) { ... }
}

// Kullanım - okunmaz hale gelir
Hesap h = new Hesap("Ali", "Veli", "12345", null, null,
                    "VADESİZ", "TL", 1000, "S001", null, 0.0, null);
// Bu hangi parametre neydi??
```

Problemler:

- Hangi parametre ne anlama geliyor okunmuyor
- Opsiyonel alanlar için null geçmek zorundasın
- Sırayı karıştırırsan compile etse bile yanlış nesne yaratırsın
- 12'den 13'e çıkarsa tüm çağırıcıları güncellemek lazım

7.2. Builder Pattern Çözümü

```
public class Hesap {
    private final String ad;
    private final String soyad;
    private final String tcNo;
    private final String telefon; // opsiyonel
    private final String email; // opsiyonel
    private final String hesapTipi;
    private final BigDecimal bakiye;

    // Private constructor - sadece Builder'dan yaratılır
```

```

private Hesap(Builder b) {
    this.ad = b.ad;
    this.soyad = b.soyad;
    this.tcNo = b.tcNo;
    this.telefon = b.telefon;
    this.email = b.email;
    this.hesapTipi = b.hesapTipi;
    this.bakiye = b.bakiye;
}

public static Builder builder() {
    return new Builder();
}

public static class Builder {
    private String ad;
    private String soyad;
    private String tcNo;
    private String telefon;
    private String email;
    private String hesapTipi = "VADESIZ"; // default
    private BigDecimal bakiye = BigDecimal.ZERO;

    public Builder ad(String ad) { this.ad = ad; return this; }
    public Builder soyad(String soyad) { this.soyad = soyad; return this; }
    public Builder tcNo(String tcNo) { this.tcNo = tcNo; return this; }
    public Builder telefon(String t) { this.telefon = t; return this; }
    public Builder email(String e) { this.email = e; return this; }
    public Builder hesapTipi(String h) { this.hesapTipi = h; return this; }
    public Builder bakiye(BigDecimal b){ this.bakiye = b; return this; }

    public Hesap build() {
        // Validation burada yapılabilir
        if (ad == null || tcNo == null) {
            throw new IllegalStateException("Ad ve TC zorunlu");
        }
        return new Hesap(this);
    }
}
}

```

Kullanımı çok okunaklı:

```

Hesap hesap = Hesap.builder()
    .ad("Ali")
    .soyad("Veli")
    .tcNo("12345678901")
    .email("ali@example.com")
    .bakiye(new BigDecimal("1000"))
    // telefon koymadık, default null
    // hesapTipi koymadık, default "VADESIZ"
    .build();

```

7.3. Lombok ile Kısaltma

Bu builder kodunu elle yazmak sıkıcı. Lombok'un @Builder annotation'ı tüm bu kodu derleme zamanında üretir:

```
@Builder
@Getter
public class Hesap {
    private String ad;
    private String soyad;
    private String tcNo;
    private String telefon;
    private String email;

    @Builder.Default
    private String hesapTipi = "VADESIZ"; // default değer

    @Builder.Default
    private BigDecimal bakiye = BigDecimal.ZERO;
}

// Kullanım aynı
Hesap h = Hesap.builder()
    .ad("Ali")
    .soyad("Veli")
    .tcNo("12345")
    .build();
```

❑ **Akılda Kalması Gereken: Builder pattern'in faydaları:**

- 1) Okunabilirlik — her alan adıyla görünür**
- 2) Opsiyonellik — istemediğin alanı yazmazsın**
- 3) Immutability — build() sonrası nesne değişmez, final field'lar**
- 4) Default değerler — Builder'da set edebilirsin**
- 5) Validation — build() içinde kontrol yaparsın**

Modern Java'da Lombok'un @Builder'ı standart. Java 16+ ile record'lar da benzer immutability sağlar.

Gerçek hayattan örnekler (zaten kullandığın yapılar):

```
// HttpRequest API'sı
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com"))
    .header("Authorization", "Bearer xxx")
    .POST(HttpRequest.BodyPublishers.ofString(body))
    .build();

// StringBuilder
String sonuc = new StringBuilder()
    .append("Merhaba ")
    .append(ad)
    .append(", hoş geldin")
    .toString();
```

```
// Stream.Builder  
Stream<String> s = Stream.<String>builder()  
    .add("a")  
    .add("b")  
    .build();
```

8. Spring'i Yüzeyden Derinleştirme

Spring'i kullanıyorsun ama "nasıl çalıştığını" tam bilmiyorsun. Mülakatta bu fark edilir. Birkaç önemli iç mekanizmayı netleştirelim.

8.1. @Repository'nin Sihri: Exception Translation

Mülakatta sordum, bilmiyordun. @Component, @Service, @Controller, @Repository hepsi bean tanımlar — neden 4 ayrı annotation var?

Anlamsal ayrım (hangi katman olduğunu belirtmek) bir sebep. Ama @Repository'nin somut bir Spring sihri var:

DAO/Repository katmanında bir SQL hatası olduğunda JDBC sana SQLException (checked exception) fırlatır. Hibernate sana kendi exception'larını fırlatır (HibernateException, PersistenceException vs).

İki problem var:

- 1) SQLException checked — üst katmanlarda try-catch zinciri ya da throws zincirleri olur, kod kirlenir.
- 2) Hibernate exception'ları teknoloji bağımlı — yarın JPA'dan jOOQ'a geçtin, tüm catch'leri değiştirmen gerekir.

Spring'de @Repository ile ne yapıyor? Otomatik exception translation:

```
@Repository
public class HesapRepo {
    public Hesap bul(String id) {
        // Hibernate burada NonUniqueResultException atabilir
        // veya JDBC SQLException atabilir
    }
}

// Servis katmanında ne yakalarsın?
@Service
public class HesapServisi {
    @Autowired private HesapRepo repo;

    public Hesap getir(String id) {
        try {
            return repo.bul(id);
        } catch (DataAccessException e) { // Spring'in ortak sınıfı
            // SQLException de Hibernate exception da DataAccessException'a
            // çevrildi
            throw new HesapBulunamadiException(id, e);
        }
    }
}
```

Spring'in DataAccessException hiyerarşisi:

- DuplicateKeyException — unique constraint ihlali
- DataIntegrityViolationException — foreign key, not null vs.
- EmptyResultDataAccessException — sonuç beklenirken yok
- OptimisticLockingFailureException — version çakışması
- QueryTimeoutException — sorgu timeout

Hepsi DataAccessException extends RuntimeException. Yani:

- Unchecked — throws yazmana gerek yok
- Teknoloji bağımsız — JPA → JDBC geçişinde catch'ler aynı kalır
- Anlamlı — "hangi tür hata" net belli

□ **Akılda Kalması Gereken: @Repository = @Component + automatic exception translation. Bu sadece anlamsal değil, somut bir Spring özelliği. Mülakatta bunu söylersen "Spring'i sadece kullanıyor" değil, "Spring'i anlıyor" sınıfına geçersin.**

8.2. Bean Lifecycle — Bean Doğuyor, Yaşıyor, Ölüyor

Spring uygulaması ayağa kalkarken her @Service, @Repository, @Component için bir nesne yaratır. Bunun bir akışı var:

#	Aşama	Ne Olur
1	Instantiation	Spring constructor'ı çağırır, nesneyi yaratır
2	Dependency Injection	@Autowired field'lar / setter'lar / constructor parametreleri set edilir
3	Aware interface'leri	BeanNameAware, ApplicationContextAware vs callback'leri çağırılır
4	@PostConstruct	Bean tamamen hazır, init kodun çalışır
5	BeanPostProcessors	Proxy'ler oluşturulur (@Transactional, @Async için)
6	Hazır	Bean kullanıma hazır, istek geldiğinde metotları çağırılır
7	@PreDestroy	Uygulama kapanırken cleanup kodun çalışır

@PostConstruct kullanımı (banka örneği):

```
@Service
public class ExchangeRateServisi {

    private Map<String, BigDecimal> kurlar = new ConcurrentHashMap<>();

    // Bean ayağa kalkarken bir kez çalışır
    @PostConstruct
    public void init() {
        // Uygulama başlarken kurları yükle
        log.info("Kurlar yükleniyor...");
        kurlar = merkezBankasiApi.kurlarGetir();
    }
}
```

```

        log.info("{} adet kur yüklendi", kurlar.size());
    }

    @Scheduled(fixedDelay = 60000) // Her dakika güncelle
    public void kurlariGuncelle() {
        kurlar = merkezBankasiApi.kurlarGetir();
    }

    public BigDecimal kurAl(String paraBirimi) {
        return kurlar.get(paraBirimi);
    }
}

```

8.3. Bean Scope'ları — Aynı Bean Kaç Tane Olacak?

Scope	Davranış	Ne Zaman
singleton (default)	Tüm uygulamada TEK instance. Tüm thread'ler aynı Bean servisi için kullanılır.	WS, RS, API, repository'ler, controller'lar.
prototype	Her @Autowired veya getBean() çağrısı için YENİ instance, mutable cache değil paylaşılmamalı olanlar.	Stateful Bean'lar.
request	Her HTTP isteği için yeni instance, isteğe göre spesifik veriler (user info, request ID).	Request-specific veriler (user info, request ID).
session	Her HTTP session için bir instance.	Kullanıcı oturumuna özel veriler.

⚠ Dikkat: Default singleton olduğu için servislerinde instance field tutmaktan kaçın. Aynı service nesnesi binlerce thread tarafından kullanılır. Field'a yazılan veri thread'ler arasında karışır. İstisna: cache gibi thread-safe yapılar (ConcurrentHashMap).

9. REST API ve HTTP Status Code'ları

Bu soruyu mülakatta geçtin ama temel REST bilgisi banka mülakatlarında her zaman çıkar. Hızlı bir referans:

9.1. RESTful API Tasarımının Temel Prensipleri

1) Resource bazlı URL'ler — fiil değil, isim kullan:

```
// ❌ Kötü - fiil var
POST /createUser
POST /deleteAccount/123
GET /getUserById?id=5

// ✅ İyi - resource ve HTTP metot
POST /users // Yeni kullanıcı yarat
GET /users/5 // 5 ID'li kullanıcı
DELETE /accounts/123 // 123 ID'li hesap sil
GET /accounts/123/transactions // Hesabın hareketleri
```

2) HTTP metotlarını doğru kullan:

Metot	Anlamı	Idempotent?	Banka Örneği
GET	Veri oku, hiçbir şey değiştirme	Evet	GET /accounts/123 (hesap görüntüle)
POST	Yeni kaynak yarat	Hayır	POST /transfers (havale yap)
PUT	Kaynağı tamamen güncelle/yenile	Evet	PUT /customers/5 (müşteri güncelle)
PATCH	Kaynağın kısmi güncelleme	Genelde evet	PATCH /accounts/123 (sadece limit güncelle)
DELETE	Kaynağı sil	Evet	DELETE /cards/456 (kart sil)

Idempotent ne demek? Aynı isteği 1 kez veya 100 kez göndermek aynı sonucu üretir. POST /transfers idempotent değildir — 100 kez gönderirsen 100 havale yapılır. GET, PUT, DELETE doğaları gereği idempotent olmalıdır.

3) Stateless: Her istek kendi başına anlamlı olmalı, server session tutmamalı (JWT gibi token'larla).

4) Versiyonlama: /api/v1/accounts — geriye uyumluluk için.

5) Anlamlı status code'lar — aşağıda.

9.2. HTTP Status Code'ları — Banka Senaryolarıyla

2xx — Başarılı

200 OK	Başarılı, response body var	GET /accounts/123 → hesap bilgisi döndü
201 Created	Yeni kaynak yaratıldı, Location header var	POST /transfers → yeni havale yaratıldı, ID döndü

204 No Content	Başarılı ama body yok	DELETE /cards/456 → silindi, dönecek bir şey yok
-----------------------	-----------------------	--

4xx — Client Hatası (Senin yanlış istek attın)

400 Bad Request	İstek bozuk, validation hatası	POST /transfers ile negatif miktar gönderdin
401 Unauthorized	Kimliğin yok / yanlış	Token yok veya expired, login ol
403 Forbidden	Kimliğin var ama bu işleme yetkin yok	Başka müşterinin hesabını görmeye çalıştın
404 Not Found	Kaynak yok	GET /accounts/999999 → hesap yok
409 Conflict	İstek kaynağın mevcut durumuyla çakışıyor	Aynı hesabı 2. kez kapatmaya çalıştın, zaten kapalı
422 Unprocessable Entity	Syntax doğru ama iş kurallarına aykırı	IBAN formatı doğru ama hesap pasif
429 Too Many Requests	Rate limit aştın	Saniyede 100 istek attın, sınır 50

5xx — Server Hatası (Bizde sorun var)

500 Internal Server Error	Beklenmeyen sunucu hatası	NullPointerException patladı, kod bug'lı
502 Bad Gateway	Backend'in arkasındaki servis cevap vermedi	Kapı bankası API'si down
503 Service Unavailable	Geçici olarak hizmet veremiyoruz	Maintenance, ya da aşırı yük, sonra dene
504 Gateway Timeout	Backend zamanında cevap vermedi	Karşı banka 30sn'de cevap vermedi

□ Akılda Kalması Gereken: 401 vs 403 farkını net bil — mülakatçı sever:
401 Unauthorized = "Sen kimsin? Kimliğini gösteremedin"
403 Forbidden = "Kim olduğunu biliyorum ama bu işi yapamazsın"

10. Production Incident Yönetimi — Banka Kültürü

Mülakatın en kritik anı buydu: "Para gitti karşıya ulaşmadı." Sen "manuel SQL ile düzeltirim" dedin. Bu bankada büyük bir kırmızı bayrak. Burada doğru protokolü öğreneceksin.

10.1. Banka Kültürünün Farkı

Normal şirkette bir prod bug fark ettiğinde geliştirici hızlıca düzeltir, deploy eder, devam eder. Bankada bu mümkün değil.

Çünkü bankada:

- Her işlemin para etkisi var — hata maliyetli
- BDDK denetimleri var — audit trail zorunlu
- Compliance/risk ekipleri var — onlardan onay alınmadan müdahale olmaz
- Yetki ayrımı (segregation of duties) — bir kişi tek başına kritik aksiyon alamaz
- Dual control — kritik işler için 2 kişi onayı şart

⚠ Dikkat: Mülakatta asla "tek başıma SQL ile düzeltirim" demeysin. Hangi pozisyonda olursan ol, bu cevap seni eler. Doğru cevap: "Yöneticime ve risk ekibine bildirir, onaylı süreçle düzeltme yaparım."

10.2. İlk 15 Dakika — Doğru Sıralama

□ Gerçek Hayat Senaryosu:

Üretimde para hareketi sorunu fark ettin. Senin yazdığın kod, senin sorumluluğun. Çağrı merkezi şikayetleri geliyor. Ne yaparsın?

Adım 1: SAKİN OL ve SOLO DAVRANMA (0-2 dakika)

İlk içgüdü "hemen düzelteyim" olacak. YAPMA. Bankada sessiz solo müdahale işten atılma sebebidir. İlk yapacağın şey ekibe haber vermek.

Adım 2: BİLDİR (2-5 dakika)

Bir Slack/Teams kanalına net bir mesaj at:

```
@channel INCIDENT - Üretimde para hareketi sorunu
- Etki: EFT yapan müşteriler "para gitti ulaşmadı" diyor
- Başlangıç: ~10:15
- Tahmini etki: son 30 dakikada yapılan EFT'ler
- Sorumlu: ben (Batur)
- İlk aksiyon: incident channel açıyorum
- Çağrı merkezini bilgilendiriyoruz
```

Adım 3: KANAMA DURDURMA (5-10 dakika)

Root cause aramaktan önce hasarı durdur. Yeni etkilenecek müşteri yaratma. Seçenekler:

- Feature flag: yeni özelliği geçici kapat
- Rollback: son deploy'u geri al
- Circuit breaker'ı aç: ilgili akışı geçici durdur
- Endpoint'i devre dışı bırak: maintenance moduna al

Müşteri "şu an EFT yapamıyorum" diye şikayet etmesi, "param gitti ama ulaşmadı" demesinden 1000 kat daha iyi.

Adım 4: WAR ROOM AÇ (10-15 dakika)

Tek başına çözemezsin, çözmemeli. War room katılımcıları:

- Senin ekibinden bir kıdemli — kod incelemesi
- DBA — DB tarafından bakış
- DevOps/SRE — altyapı, deploy, log access
- Business analyst / Product — iş etkisi, müşteri iletişimi
- Risk/Compliance — kritik finansal etki varsa zorunlu
- Yöneticin — kararları alacak kişi

10.3. Root Cause Analizi — Hangi Log'a Bakılır?

Bankada genelde şu loglar/araçlar olur:

- Application log — senin loggerinin yazdığı (logback, log4j)
- Distributed tracing — Zipkin, Jaeger; bir request'in tüm yolculuğu, correlation ID ile takip
- APM — Dynatrace, AppDynamics, New Relic; metrikler, slow query, GC pause
- Integration log — dış sistemlere gönderilen istekler, alınan cevaplar
- DB log — slow query log, deadlock log, transaction log
- Kafka/MQ monitoring — mesaj kuyrukları sağlıklı mı, lag var mı

"Para gitti ulaşmadı" senaryosunda kontrol sırası:

- 1) Senin DB: hesaptan para gerçekten düştü mü? (audit kayıtları)
- 2) Integration log: karşı bankaya çağrı atıldı mı? Cevap geldi mi?
- 3) Karşı banka log: ulaştı mı, işlem yaptı mı?
- 4) Network monitoring: o saatte timeout/packet loss var mıydı?
- 5) Application log: exception fırlatıldı mı? Hangisi?

10.4. Düzeltme — Asla Tek Başına Değil

Diyelim ki root cause bulundu: karşı bankaya gitmiş ama dönüş alınamamış, senin DB'nde de para düşmüş. Müşteriler etkilenen kayıtlar belli. Şimdi ne yaparsın?

Yanlış cevap: "Hemen SQL ile bakiyeleri düzeltirim."

Doğru süreç:

- 1) Etkilenen müşteri listesini çıkar — SELECT sorgusu ile, UPDATE yok
- 2) Listeyi risk/compliance'a gönder — etki raporu
- 3) Düzeltme planı hazırla — "hangi kayıt nasıl düzeltilecek" doküman
- 4) Yöneticiden ve risk ekibinden yazılı onay al
- 5) Dual control ile düzeltme uygula — sen SQL yaz, başka biri review etsin, ikinizin onayı ile production'da çalışsın
- 6) Her UPDATE log'lansın — audit trail için
- 7) Müşterilere iletişim — "Sorun çözüldü, paranız hesabınızda" SMS
- 8) Incident kapatılır

10.5. Post-Mortem — Bir Daha Olmaması İçin

Incident bittikten sonra (genelde 24-48 saat içinde) blameless post-mortem yapılır. Amaç: kimi suçlayalım değil, sistemde neyi değiştirelim ki bir daha olmasın.

Post-mortem dokümanı şunları içerir:

- Timeline: dakika dakika ne oldu, kim ne yaptı
- Etki: kaç müşteri, ne kadar para, ne kadar süre
- Root cause: 5 Why analizi ile en derin sebep
- Contributing factors: hangi savunmalar başarısız oldu
- Aksiyon maddeleri: somut, sorumlu kişisi olan, tarihi olan
- Önleyici tedbirler: monitoring, test, kod review, training

□ İyi Pratik: Örnek aksiyonlar: "EFT akışına idempotency key eklenecek (sorumlu: Batur, tarih: 30 gün), outbox pattern'e geçilecek, integration timeout 5sn'den 30sn'ye çıkarılacak, reconciliation job'a karşı banka entegrasyonu eklenecek."

□ **Akılda Kalması Gereken: Banka incident management özet:**

- 1) Solo davranma — önce bildir, sonra çöz
- 2) Önce kanamayı durdur — root cause sonra
- 3) War room — kolektif çözüm
- 4) Asla tek başına SQL çalıştırma — dual control, audit, onay
- 5) Blameless post-mortem — kişi değil sistem
- 6) Aksiyonlar takip edilir — bir daha olmasın

11. Mülakatta Anlatabileceğin Hikayeler

Mülakatta teorik bilgi kadar somut deneyim hikayeleri değerlidir. ARGUS deneyimin aslında çok zengin — ama "HR projesi" deyince mülakatçı tam bağlantı kuramayabilir. Bu bölümde sahip olduğun gerçek deneyimleri banka diline çevirmeyi öğreneceksin.

11.1. STAR Yöntemi — Hikayeni Yapılandır

Her hikaye 4 parçaya bölünür:

S - Situation (Durum): Hangi proje, ne zaman, ne yapıyordun

T - Task (Görev): Senin sorumluluğun neydi

A - Action (Aksiyon): Sen ne yaptın (teknik detay)

R - Result (Sonuç): Ne elde edildi, ne öğrendin

11.2. Senin Hikayelerin (ARGUS'tan)

Hikaye 1: Karmaşık İş Kuralı Bug'ı (Tosyalı yarım gün izin)

S: ARGUS HR uygulamasında belirli bir müşteri (Tosyalı) için yarım gün izinler, ardışık izin kontrol kurallarına takılıyordu. Sistemin tüm müşteriler için aynı kontrolü uygulaması gerekiyordu ama bu müşteride istisna gerekiyordu.

T: Mevcut kontrol mantığını bozmadan istisna eklemek.

A: WfizinCommand sınıfında iki farklı yerde aynı kontrolün yapıldığını tespit ettim (tarihKontrolWfizin ve tarihKontrollznper). İkisini ayrı ayrı patch'lemek yerine ortak bir guard pattern oluşturdum. Edge case'leri (yarım günün çoklu izin arasına düşmesi) TODO olarak dokümente ettim.

R: Hot fix prod'a çıktı, müşteri sorunu çözüldü. Ortaya bir sonraki sprint'te alınacak teknik borç çıktı, gözardı edilmedi.

Bunu mülakatta nasıl satarsın?

□ İyi Pratik: "Bir müşteri-specific bug'ı vardı. Çözüm yaparken sadece patch atmak yerine kod duplication'ı fark edip ortak bir pattern oluşturdum, hem o bug çözüldü hem de ileride benzer müşteri istisnalarına hazır olduk. Tespit ettiğim edge case'i TODO ile dokümente edip teknik borç olarak takip ettik." — Bu cümleler senior bir mühendis sesi.

Hikaye 2: Production Bug — Portal İzin Tipi Görünmüyor

S: Portal üzerinden izin talep eden kullanıcılar, formu hatayla doldurup tekrar denediklerinde "portalda gösterilmeyecek" işaretlenmiş izin tiplerinin geri geldiğini görüyordu.

T: Root cause analizi ve düzeltme.

A: Kodu izleyerek setParametrikDegerler metodunun çağrı zincirini takip ettim. Yöneticinin kendi personel kaydından izin tipi yüklendiğini ama seçilen personelinin yüklenmesi gerektiğini fark ettim. IKReusable.loadPersicByIsyeriSicil ile doğru personeli yükleyip parametre yüklemesini ona göre yaptım. Ayrıca insert hata dönüş yollarında setParameter çağrılarının eksik olduğunu fark ettim ve onları da ekledim.

R: Sorun çözüldü, ek olarak benzer yan etkiler önlendi.

Banka diline çevirisi:

□ İyi Pratik: "Bir UI bug'ı vardı ama altında veri yükleme akışında yanlış kullanıcı bağlamı (wrong context) sorunu vardı. Stack trace yerine kodu okuyarak business logic seviyesinde neden olduğunu tespit ettim. Bug fix sırasında benzer pattern hatalarını da yakalayıp düzelttim. Bu, defensive programming yaklaşımımın bir örneği."

Hikaye 3: Performans/Bulk Upload (Eğitim Toplu Yükleme)

S: Personel eğitim geçmişlerini tek tek girmek operasyonel olarak imkansızdı. Toplu Excel yükleme ekranı talep edildi.

T: Yeni bir bulk upload akışı tasarla ve uygula.

A: Mevcut bir bulk upload pattern'i (excel_tipi=4, VerexlCommand) inceleyip ARGUS'un kalıbına uydurdum. Yeni Command, Form ve JSP'leri oluşturdum. Geliştirme sırasında çıkan NPE'leri (screen mapping, JS validation, help button) tek tek çözdüm.

R: Operasyon ekibinin günler süren manuel data entry'si 5 dakikalık bir işlem oldu.

Banka diline çevirisi:

□ İyi Pratik: "Manuel operasyonel yükü ortadan kaldıracak bir bulk processing akışı tasarladım. Sıfırdan yazmak yerine mevcut bir pattern'i extend ederek tutarlılık sağladım. Hem geliştirme süresini kısalttım hem de operasyonel verimliliği artırdım. Banka'nın günlük raporlama, hesap toplu işlemler, vs. ihtiyaçlarına aynı yaklaşımı uygulayabilirim."

11.3. Mülakatta Sorulacak Soru ve Cevap Şablonları

"Bana en zorlu prod bug'ını anlat"

Hikaye 1 veya 2'yi kullan. Vurgu: sistematik debug yaklaşımın, side effect'leri yakalama, fix dışında ek değer üretme.

"Bir teknik borç ile nasıl başa çıkarsın?"

Hikaye 1'deki TODO yaklaşımını anlat. "Anlık çözüm üretirim ama edge case'leri dokümente eder, sprint planning'de teknik borç olarak ekibe sunarım."

"Mevcut işinden neden ayrılmak istiyorsun?"

Negatif konuşma. Pozitif framing: "HR domain'inde derinleştirdim. Şimdi yeni bir domain'de büyümek istiyorum. Bankacılık daha kritik sistemler, daha yüksek ölçek ve daha karmaşık iş kuralları sunuyor. Akbank'ın mühendislik kültürü, modern stack'i ve ölçeği beni heyecanlandırıyor."

"5 yıl sonra kendini nerede görüyorsun?"

Klişe "tech lead olmak istiyorum" yerine: "Önümüzdeki 2 yıl bankacılık domain'inde ve dağıtık sistemlerde uzmanlaşmak istiyorum. Sonrasında ya mimari/teknik liderlik, ya da ödeme sistemleri gibi kritik bir alanda derinleşmiş bir uzman olmak istiyorum. Akbank'ta bunun yolu nedir, sizden öğrenmek isterim."

"Sana zor bir teknik kararı senin alman istense ne yaparsın?"

"Tek başıma karar vermem. Önce trade-off'ları yazılı olarak çıkarırım — performans, maliyet, geliştirme süresi, bakım kolaylığı, risk. Bunları ekiple ve gerekirse mimari ekiple paylaşıyorum. POC yapılması gerekiyorsa yaparım. Karar dokümana bağlanır, rationale not edilir."

11.4. Son Tavsiyeler

- 1) Dürüst ol. Bilmediğin bir şey gelirse "bunu detaylı bilmiyorum ama duyduğum kadarıyla şu var" diyebilirsin. Banka mülakatçıları yalandan çok sezgilidir.
- 2) Düşünme sürecini sesli yap. Cevabı anında bilmiyorsan "şu açıdan düşüneyim... öyleyse şu da olabilir" diye düşünce zincirini görmelerine izin ver. Cevap bulamasan bile düşünme şeklin değerli.
- 3) Banka kelimelerine alış. EFT, havale, IBAN, mutabakat, kambiyo, mevduat, kredi limiti, KKB, MERNİS, takas, tasfiye, BDDK gibi terimleri rahatça kullan.
- 4) Geri soru sor. Mülakat çift yönlüdür. "Sizin ekibinizdeki tech stack ne? Hangi metodoloji ile çalışıyorsunuz? Yeni başlayan biri ilk 30-60-90 günde ne yapar? Mentorship var mı? Production deploy süreçleri nasıl?" gibi sorular hem sana bilgi verir, hem de senin profesyonel olgunluğunu gösterir.
- 5) Senin avantajların: 3+ yıllık enterprise Java deneyimi, prod debug deneyimi, performans/transaction sorunlarıyla yüzleşmiş olmak, multi-tenant müşteri yönetimi, büyük kod tabanlarında çalışma deneyimi. Bunları öne çıkar. Junior olmadığını hissettir.

Son Söz

Bu dokümandaki konuların hepsini bir gecede mükemmel öğrenmen gerekmiyor. Amacı, mülakat olduğunda bu konular geldiğinde "yabancı" gelmemesi. Soru gelince kafanda bir resim canlansın, başlangıç bir cevap üretebilesin.

Hatırla: mülakatta mükemmel cevap beklenmiyor. Düşünme şeklin, dürüstlüğün ve öğrenmeye açıklığın bilgin kadar değerli. ARGUS'ta yaptığın gerçek işler senin en büyük cephanen. Bunu bilen, mantıklı düşünen biri olarak girdiğinde işin yarısı zaten halledilmiş olur.

Başarılar Batur.